

Revise a pull request

Week 8 · a reference, and a Friday activity · about 20 minutes

Brian C. Keegan, Ph.D. · Department of Information Science, University of Colorado Boulder

October 9, 2026

What you'll do

Your pull request has a review, and the reviewer asked for changes. That's a normal step on the way to merging, and it doesn't mean your pull request was rejected.

You make the changes on the same pull request. A pull request follows its branch, so each commit you add to the branch shows up in the pull request right away, next to the comment it answers. A new pull request would lose that conversation, and your reviewer would have to start over.

You'll do all of it in your browser, with three tools. The box on the right matches each kind of comment to its tool.

The screenshots come from textbook pull request #228, a practice pull request that the instructor set up. Its base branch, `demo/week08-base`, is a copy of `main`. So wherever a screenshot says `demo/week08-base`, your pull request says `main`.

Which tool for which comment

A comment with a suggestion box	Step 2: Apply suggestion
A comment that asks for a change in words	Step 3: Edit file
The notice <i>This branch has conflicts</i>	Step 4: Resolve conflicts

Please don't

- open a new pull request, or close this one: the conversation lives here;
- click **Merge pull request**: the instructor merges after Friday's code review;
- change a file in `notebooks/` or `images/`: scripts make those;
- fix things the review didn't ask about. Spotted another problem? Save it for a new pull request.

In class

Work in pairs, on one partner's pull request. One of you works in the browser while the other reads this handout aloud. Then switch.

1 · Read the whole review

1. Sign in to GitHub, and open your pull request (<https://github.com/cuinfoScience/Web-Data-Science-Book/pulls>). To list only yours, add `author:@me` to the search box above the list.
2. On the **Conversation** tab, read the review's summary first, then each comment under it. A comment on a line shows that line above it. If GitHub hides part of a long conversation behind a link, click it, so you don't miss a comment.
3. Make a list: one item for each change the review asks for, with its tool from the box on page 1. #228's review asks for two changes: fix a typo (a suggestion), and add `--verbose` to the `curl` line (a comment). Its conflict makes a third.
4. If a comment doesn't make sense to you, or you disagree with it, reply in its thread before you change anything. Say what you think, and why. Questions and polite disagreement are a normal part of a review.

2 · A suggestion: Apply suggestion

When a reviewer writes out the exact new words, they put them in a **suggestion** box: the old line in red, the new line in green, and a button under them. You can accept it in a few clicks.

1. Read the green line first. Applying it replaces the red line exactly as shown. If the green line isn't quite right, don't apply it: make the change yourself with **Edit file** (step 3), and say so in the thread.
2. Click **Apply suggestion**. A box opens under the button. Write a commit message that says what changed, for example `Ch. 5: fix the spelling of Network`, and click **Commit changes**.
3. The comment now says the suggestion was committed, and the pull request shows one more commit. That's all: the change is already on your branch, with nothing else to save.

Several suggestions? Click **Add suggestion to batch** on each one, then commit them all at once from the button that

counts them. You get one commit instead of several.

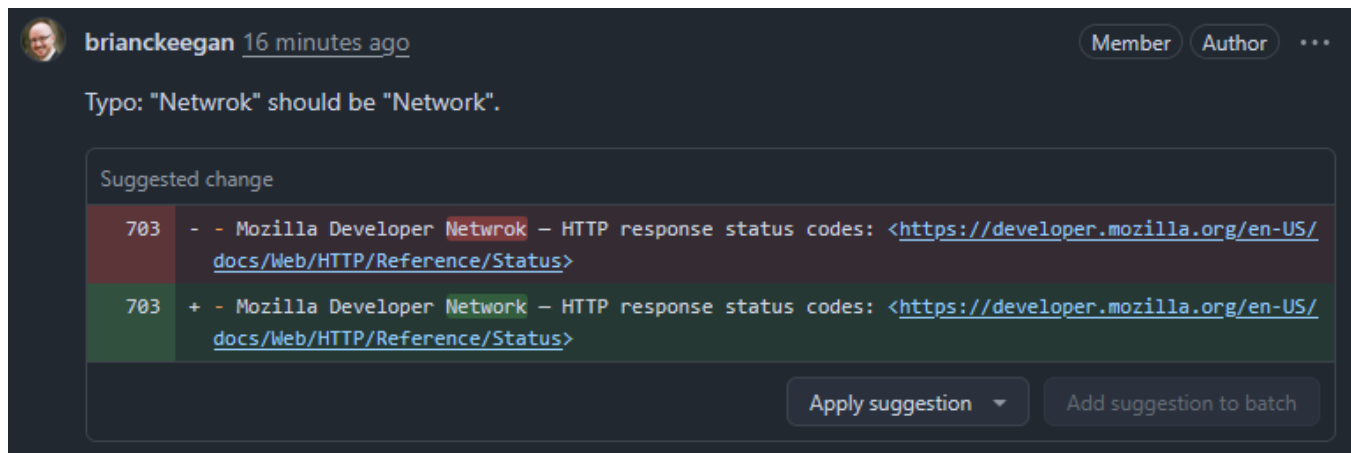


Figure 1. A suggestion on #228, captured in October 2026: the old line in red, the new line in green, and **Apply suggestion** under them.

3 • A comment: Edit file on your branch

A comment without a suggestion box tells you in words what to change, so you make the change yourself.

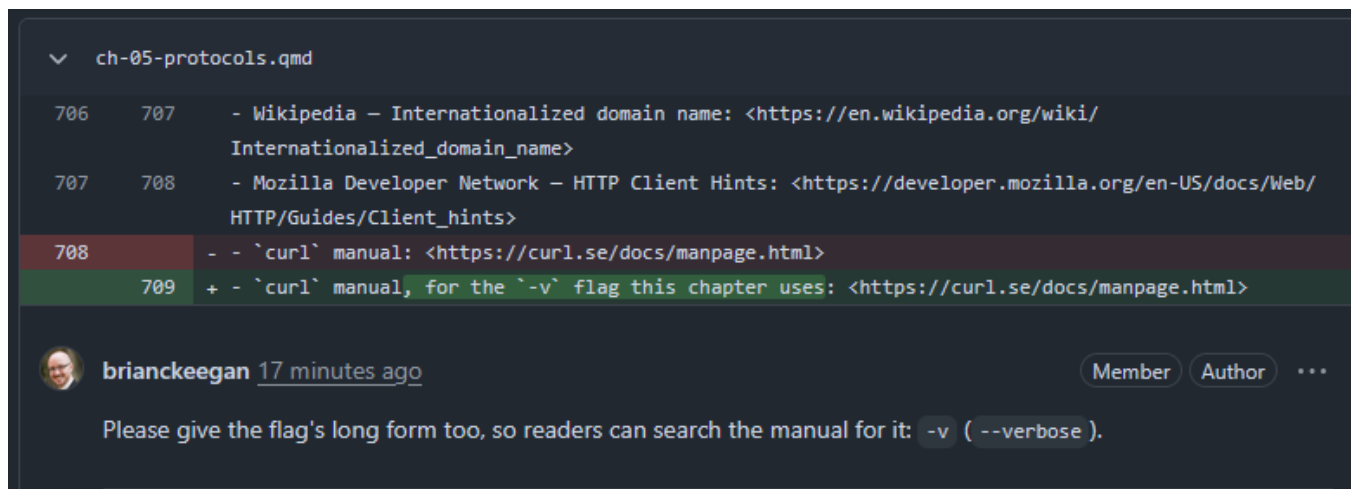


Figure 2. A comment on #228's curl line, in **Files changed**, captured in October 2026. It asks for a change in words, so you make it yourself.

1. Click **Files changed**. Find the file, click its **...** menu at the right of the file's name, and click **Edit file**.
2. Before you type anything, check the top of the editor. It should name your pull request's branch, not main. (#228's branch is demo/week08-revise.) It's easy to land in the wrong copy of the file here. If the editor says main, or asks you to fork the repository, you opened the file from the book's page, not from your pull request: close the editor and start again from your pull request's **Files changed**.
3. Press Ctrl+F (Cmd+F on a Mac) in the editor to find the line. Make the change that the comment asks for, and nothing else. Click **Preview** to compare the red and green lines.
4. Click **Commit changes...**, which opens a dialog. Write a commit message, and leave **Commit directly to the your-branch branch** selected. Don't make a new branch: a new branch would need a new pull request. Then click **Commit changes** in the dialog.

4 • A conflict: Resolve conflicts

If main changed the same lines as your pull request after you opened it, the bottom of the **Conversation** tab says *This branch has conflicts that must be resolved*. You didn't do anything wrong: another change reached main first, and GitHub can't choose between the two versions, so it asks you to. Resolve it when a review asks you to.

1. Click **Resolve conflicts**. GitHub opens an editor and counts the conflicts at the top. Each conflict sits between three marker lines:

```

<<<<<< demo/week08-reverse
- `curl` manual, for the `-v` flag this chapter uses: <https://curl.se/docs/manpage.html>
=====
- `curl` manual, for `-v` and `-I`: <https://curl.se/docs/manpage.html>
>>>>>> demo/week08-base

```

- The first part is your branch's version, which GitHub calls the *current change*. The second is the base's (on your pull request, main's), the *incoming change*. If those names seem backwards, remember that *current* means yours. Above the two parts, three links each keep one side or both. Click **Accept both changes**, then edit the two lines into one line that keeps what each side meant. For #228, the base added `-I`, and the review asked for `--verbose`:

```

- `curl` manual, for `-v` (`--verbose`) and `-I`: <https://curl.se/docs/manpage.html>

```

- Check that no marker line (`<<<`, `===`, `>>>`) is left. **Mark as resolved**, at the top right, stays gray until then. Click it, then **Commit merge**. Your pull request gains a commit that merges the base into your branch. That's expected.

If the button is gray and says the conflict is *too complex to resolve in the web editor*, stop. The web editor can't handle every conflict, and this one needs a different tool. Comment on your pull request that you need help with a conflict, and the instructor will resolve it.

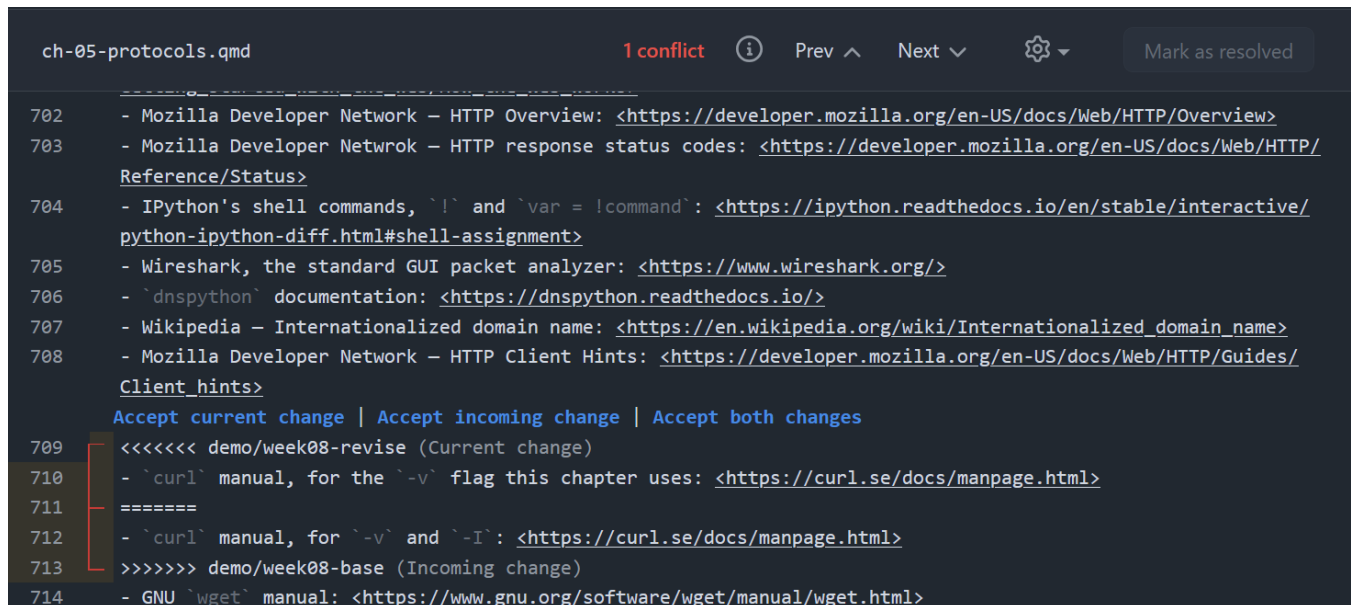


Figure 3. GitHub's conflict editor on #228, captured in October 2026: the two versions of the line between the marker lines, the three **Accept** links above them, and **Mark as resolved**, gray until the conflict is resolved.

5 • Answer the review, and ask again

- Reply in each thread with what you did, for example Done: added the long form. Click **Resolve conversation** under a thread once its change is made. Leave a thread open if you disagreed and are waiting for an answer.
- In the right-hand sidebar, under **Reviewers**, click the circular arrows beside your reviewer's name, **Re-request review**. Your reviewer gets a notice that your pull request is ready for another look. The pull request keeps saying *Changes requested* until they approve it, even after you've made every change.
- Watch the checks at the bottom of the **Conversation** tab. They run again after each commit. A red **Notebook sync** is expected for a change made in the browser, and you don't need to fix it. A red **Render** is yours to fix: click **Details**, find the first ERROR line, and fix it with step 3.

If your pull request came from your fork's main branch

You can tell because it says *wants to merge into cuinfoscience:main from your-name:main*. Every commit you make on your fork's main joins this pull request, so commit only the review's changes there until it merges, and start your next change on a new branch. If your fork offers **Sync fork**, click **Update branch**, never **Discard commits**: discarding deletes your pull request's commits.

When something goes wrong

Find what you see below. The arrow under it says what to do. Each of these can be fixed, and none of them loses your work.

- **Apply suggestion** is gray, or the comment says *Outdated*.
 - The line changed after the reviewer commented, so the suggestion can't be applied. Make the change yourself with **Edit file** (step 3).
- The editor names `main`, or GitHub asks you to fork the repository.
 - You opened the file from the book, not from your pull request. Close the editor without committing: nothing changes until you commit. Start step 3 again from **Files changed**.
- Your commit opened a new pull request.
 - You chose **Create a new branch**. Close the new pull request, with a comment that names your first one. Then make the change again on the first pull request's branch.
- **Resolve conflicts** is gray.
 - The conflict is too complex for the web editor. Comment on your pull request, and the instructor will resolve it.
- Your pull request still says *Changes requested* after you've made every change.
 - That label stays until your reviewer approves. Click **Re-request review** (step 5), then give them time.
- The checks say a workflow is *awaiting approval*.
 - Checks on a pull request from a fork wait until the instructor approves them. You don't need to do anything.
- You closed your pull request by mistake.
 - Click **Reopen pull request**, at the bottom of the **Conversation** tab. Its branch and commits are still there.

What happens next. When your reviewer approves, the instructor merges your pull request after Friday's code review. If more comments arrive before then, answer them the same way.